

1 · Function with a formula

```
def function_name(param1, param2, param3):
    step1 = <formula using params>
    step2 = <formula using params>
    result = <formula using step1, step2>
    return result

param1 = float(input("Enter ...: "))
param2 = float(input("Enter ...: "))
param3 = float(input("Enter ...: "))
answer = function_name(param1, param2, param3)
print()
print("function name:", round(answer, #), "the unit")
```

3 · Loop with totals & counter

```
total = 0.0          # running total, starts at 0
total_cost = 0.0     # second running total
count = 0            # counter, starts at 0

for i in range(1, N + 1):    # "day 1", "day 2"...
    value = float(input("Enter value for item "
                        + str(i) + ": "))

    if value <= <limit>:
        cost = value * <rate1>
    else:
        cost = value * <rate2>
        count = count + 1    # count matches

    total = total + value    # add to totals
    total_cost = total_cost + cost

print()
print("Total:", total)
print("Count:", count)
print("Total cost: $" + str(round(total_cost, 2)))
# average = total / N (if asked for an average)
```

4 · Nested-loop pattern

```
n = int(input("Enter a positive integer: "))
print()

for row in range(n):          # outer = rows
    for col in range(<width>): # inner = items
        print(<what to print>, end=" ")
    print()                   # next line
```

2 · Conditions / categories

```
def function_name(value, flag):
    # 1. Special cases that skip everything else
    if <special case>:
        return <fixed value>

    # 2. Pick the category, lowest range first
    if value <= <limit1>:
        result = <amount1>
    elif value <= <limit2>:
        result = <amount2>
    else:
        result = <amount3>

    # 3. Extra rules (surcharge, discount...)
    if flag == "yes":
        result = result + <extra>

    return result

value = int(input("Enter ...: "))
flag = input("Question? ")
answer = function_name(value, flag)
print()
print("Label: $" + str(answer))
```

1 · Function with a formula

- Use the **exact function name and parameter order** from the question.
- `float(input())` for decimals, `int(input())` for whole numbers (ages, counts).
- The function must return the result; don't print inside it unless asked.
- Only `round(x, 2)` if the question says "rounded to 2 decimal places".

2 · Conditions / categories

- Handle the **exception first** and return early (e.g. under 5 always free).
- With `elif`, each condition only needs the **upper limit**.
- "Under 5" = `< 5`, "5–17" = `<= 17`, "65 or older" = `else / >= 65`.
- Compare text with `==` (`flag == "yes"`); don't use `int()` on it.
- Money: "Label: \$" + `str(answer)` (no space after the \$).

3 · Loop with totals & counter

- **Start every total at 0 before the loop**, never inside it.
- Add to totals **inside** the loop; print **after** it (no indentation).
- No `sum()`, `max()`, `min()`: add by hand with `total = total + value`.
- Max by hand: first value is the starting max, then `if value > biggest: biggest = value`.
- `range(1, N + 1)` counts 1 to N; `range(N)` counts 0 to N-1.

4 · Nested-loop shapes

Pattern	<width>	<what to print>
Square of *	n	"*"
Alternating rows 0/1	n	0 if row % 2 == 0 else 1
Alternating columns 0/1	n	col % 2
Checkerboard 0/1	n	(row + col) % 2
Triangle growing (1, 2, 3...)	row + 1	"*"
Triangle shrinking	n - row	"*"
Number triangle 1 / 1 2 / 1 2 3	row + 1	col + 1
Each row repeats its number	row + 1	row + 1

- `end=" "` keeps items on one line; empty `print()` ends the row.
- `row % 2 == 0` = even rows, for anything that alternates.
- Test with the smallest case, `n = 1` or `2`.

General exam rules

- One section per question, with a header comment.
- "Define a function" → write the function first, then the test code that calls it.
- Match the `input()` prompts and `print()` labels to the sample run.
- Test the sample input, then **another input**.
- No `sum`, `max`, `min`, `pow` unless allowed.
- Allowed: `print()`, `input()`, `int()`, `float()`, `range()`, `round()`.